Volund: An Empirical Study of Unattended Multiagent Program Optimization

Gustavo Wagon Widman
Instituto de Tecnologia e Liderança (Inteli)
São Paulo, Brazil

Laiza Ribeiro Silva
Instituto de Tecnologia e Liderança (Inteli)
São Paulo, Brazil

***Abstract*—We study an unattended multiagent procedure for optimizing one synthetic C++20 event range-statistics engine. Three separately initialized campaigns start from the same untuned indexed seed. A lead delegates repeated measurement and implementation work, while an independent reviewer reruns material evidence before endorsing hypotheses or source promotions. A frozen controller preserves exact artifacts, serializes native work, selects the first complete valid comparison, and requires reviewed task-based closure without a research time or token quota. All three campaigns completed, and their sealed endpoints passed the declared final validation. Fresh thirty-block comparisons gave transfer portfolio speedups of 2.240, 4.547, and 5.445; shifted-input speedups were 2.236, 5.186, and 5.294. One shifted family regressed: C1 sparse queries incurred approximately 9.7% higher latency. Thirteen hypotheses closed as nine supported, three contradicted, and one inconclusive, with eight source promotions. Observed campaign model tokens correspond to USD 17.08–18.23 under a dated Standard API-equivalent scenario; unreported compaction work and other coverage gaps prevent a whole-run cost bound. These are descriptive results for one task and the complete workflow, without a matched comparator or causal component claims.**

***Index Terms*—coding agents, program optimization, multiagent systems, empirical review, reproducibility**

I. Introduction

We report three unattended optimization campaigns that transformed the same untuned C++20 range-statistics engine into validated faster implementations. Each campaign combined delegated measurement and implementation with source-bound empirical review, then sealed an endpoint before a separate final assessment. The resulting transfer portfolio speedups were 2.240, 4.547, and 5.445. The retained evidence also captures negative hypotheses, a failed implementation and repair, a shifted-input regression, and incomplete resource-accounting coverage.

The contribution is an artifact-linked account of both optimization outcomes and the investigations that produced them. It connects exact candidate sources to correctness checks, paired timing measurements, hypothesis dispositions, review decisions, and direct seed-relative development trajectories. A frozen controller specifies how evidence authorizes promotion, how invalid timing can be retried without selecting a favorable score, and how a campaign reaches reviewed task-based closure. Reporting these rules alongside their observed execution makes the boundary between a proposed improvement and an accepted result inspectable.

Language-model program optimization already has direct precedents. PIE studies performance-improving C++ edits; AlphaEvolve uses program search with evaluator feedback; POLO and PerfAgent combine performance diagnosis with iterative refinement [1], [2], [3], [4]. The present study examines a specified end-to-end research workflow. A lead selects investigations and maintains campaign context; workers repeatedly measure, implement, test, and repair; a separate reviewer reruns material evidence before endorsing a claim or source promotion. A factual journal and automated controller retain evidence and corrections across these interactions.

The study asks three questions. **RQ1:** what valid seed-to-endpoint performance is observed on the prespecified transfer and shifted-input portfolios? **RQ2:** how do direct seed-relative development performance, hypotheses, repairs, nominations, and empirical reviews evolve in each campaign? **RQ3:** what elapsed time, observed model usage, native resources, and accounting gaps accompany the work? A campaign is the unit of replication; workload families, workers, hypotheses, and timing blocks are nested observations. No-gain, failed, stopped, and incomplete campaigns remain eligible outcomes under the protocol.

The task, compiler treatment, public workloads, and assessment rules are controlled, and every campaign starts from the original untuned indexed seed. This design yields descriptive evidence about the complete workflow on one constructed task. With no alternative workflow arm or matched research budget, it does not identify the effects of delegation, review, or other individual controls. The model optimizes an external program; model weights and inference algorithms are not optimization targets.

II. Related Work

Program optimization and iterative feedback. PIE separates functional conformance from runtime improvement and discusses variability in commodity-hardware measurements [1]. AlphaEvolve describes evolutionary program changes with automated feedback in a white paper [2]. POLO profiles project-level bottlenecks and uses collaborating agents for iterative rewrites [3]. PerfAgent, a preprint, supplies profiler and verifier feedback across repeated repository-level refinements [4]. These works establish relevant mechanisms under different tasks and evaluation regimes. Their gains are not expected effect sizes for the present task, and repeated refinement alone does not establish novelty.

Multiagent orchestration, review, and verification. AutoGen provides configurable agents, tool use, and programmable conversation patterns [5]. MetaGPT organizes software-

development roles through standard operating procedures and incorporates execution-based feedback [6]. ChatDev distinguishes reviewer–programmer code review from tester–programmer dynamic testing [7]. AgentCoder separates programming, test design, and test execution, while documenting that generated test oracles can themselves be wrong [8]. These are precedents for role specialization and executable feedback, not guarantees of independent or complete verification. Their code-generation results do not establish runtime gains on the present task. Here, an empirical review record binds a particular source and claim to the reviewer's own builds, correctness checks, profiles, and paired timings. We study the resulting campaign evidence without claiming a new orchestration framework or formal verification method.

Evaluation scope. SWE-Perf and GSO evaluate optimization in real repositories [9], [10]. Our constructed engine permits tighter control of its interface, invocation lifecycle, and source lineage, at the cost of much narrower external validity. Its results are not comparable leaderboard scores. Compiler-feedback experiments also caution against assuming that an iterative feedback procedure must outperform additional sampling [11]; the present design has no comparator that could settle that question.

Measurement and selection. Benchmarking research motivates repetition at appropriate levels and attention to incidental experimental choices [12], [13]. We retain paired observations and distinguish timing uncertainty from variation across campaigns. Selection-bias research motivates a separate final assessment [14]. Applying that argument to adaptive program development is a methodological inference, not a prior validation of this task, gate, or bootstrap procedure.

III. Method

A. Design and experimental unit

We froze the task, role instructions, controller, execution interfaces, measurement rules, and final assessment before the v3 campaigns. The cohort contains three serial campaigns, each with a fresh participant context and its own workspace initialized from the same original seed. Participants receive the public task and their own campaign evidence. Other campaigns and hidden assessment inputs and outcomes are excluded from their intended read scope. Exact artifacts and bindings are recorded in the supplement and source ledger.

The design team had seen preliminary implementations and outcomes before this freeze. Fresh participant contexts do not remove that researcher-level information advantage. Preliminary work remains engineering history and is excluded from v3 results, plots, and the campaign denominator.

The treatment is the complete workflow. Fixed task and evaluation conditions make this a controlled descriptive study, without randomized allocation or a matched alternative workflow. An unchanged-program A/A check is an instrumentation control. The three campaigns share the task, seed, host, model service, and design decisions; separate initialization does not establish statistical independence or identify component effects.

B. Task and baseline

The portable C++20 engine accepts arrays of events and queries through a fixed C ABI. Each query selects one stream or all streams within a half-open timestamp interval and returns exact count, signed sum, minimum, and maximum. Duplicate events count separately, input order is irrelevant, and empty results contain four zeros. The contract permits at most 1,000,000 events and 4,096 queries and fixes invalid-input handling, integer bounds, input immutability, and reentrancy. Each invocation must compute from its current arguments; persistent input-dependent indexes or answers are prohibited.

The original seed builds timestamp-sorted global and per-stream indexes, finds interval bounds by binary search, and scans matching values. It is an untuned indexed baseline. Index construction, query processing, and destruction all remain inside the measured invocation. Workers may redesign algorithms, layouts, allocation, batching, and parallel work within the portable standard-library contract. Canonical builds use C++20 at `-O3` with a fixed generic x86-64 target. Additional local tests and diagnostic builds supplement the immutable official checks; they cannot replace them. The supplement gives the exact ABI behavior and compiler treatment.

C. Roles and hypotheses

The lead and reviewer request `gpt-6.1-sol` with `high` reasoning effort; execution workers request `gpt-6-luna` with `xhigh` effort. Provider fallback is disabled. Requested identifiers, accepted effort settings, and reported routing are recorded. Immutable provider model versions are not exposed. Harness design and validation are setup work, accounted separately from the measured roles.

The lead primarily orchestrates and must delegate an actual baseline measurement and profiling investigation before a measurement-grounded bottleneck claim. It may reuse workers, request follow-up, compare alternatives, merge compatible changes, or abandon branches. Substantial measurement and implementation work is delegated. Workers can repeatedly edit, snapshot, compile, test, benchmark, profile, debug, and repair within their workspaces. Parallel edits require separate branches and explicit merge ownership. Reports link returned findings to actual source-bound executions.

Before its discriminating test, a hypothesis names the baseline hash, motivating evidence, proposed mechanism, predicted and disconfirming observations, test conditions, and correctness/resource risks. Exploratory observations are labeled. Revisions append the evidence that changed the claim. Closed hypotheses are supported, contradicted, or inconclusive under the tested conditions. Implementation defects and failed measurements are recorded separately from tests of the conceptual mechanism. There is no required number of hypotheses or improvements.

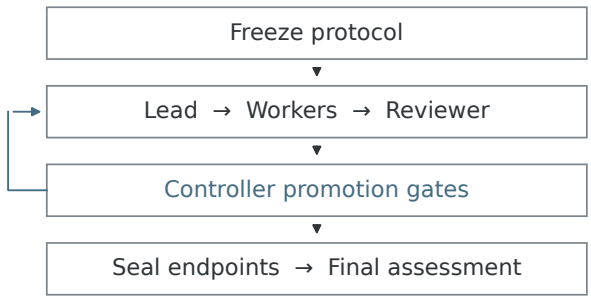


Fig. 1. Research and empirical review around a verified incumbent. Delegation, tests, repairs, and claim revision can repeat; this schematic does not prescribe a single pass. Final assessment follows all endpoint seals.

D. Independent review and promotion

Review occurs at completed hypotheses and material checkpoints, including negative and no-code findings. The reviewer starts separately from proposing-worker conversations and receives exact sources, claims, reports, and evidence pointers. For promotion, it inspects the source/diff, rebuilds under official settings, reruns correctness and resource checks, collects its own paired comparison against the incumbent, and reruns and inspects relevant profiles or traces. New negative conclusions also require direct checks of their material evidence.

The mandatory Memcheck suite spans all six public diagnostic families. The reviewer also inspects candidate-specific thresholds and materially different allocation/control-flow paths, requests relevant supplemental probes, and records unresolved coverage. Added tests remain archived. The reviewer can approve, reject, narrow a claim, or request work. Hypothesis disposition and code promotion are separate decisions. A reviewer who contributes an implementation fix becomes an optimization contributor; the resulting source requires an uncontaminated reviewer.

Exploratory branches remain separate from the verified incumbent. Nomination captures an immutable exact source, parent, build treatment, report, and evidence manifest. Repairs produce linked new sources. Promotion requires the reviewer's recommendation and every semantic, resource, integrity, and performance gate. Neutral or slower groundwork may remain exploratory.

An immutable review plan binds the source and reference across builds and reviewers. Its first complete valid official comparison in append order supplies the gate; an unfavorable valid result cannot be replaced. Required correctness/resource evidence must bind to the selected comparison's actual build. Confirmed official semantic or definite Memcheck errors disqualify that exact source under the plan; a passing rerun cannot erase them.

E. Native measurement

Six public workload families cover small build-dominated batches, sparse ranges, dense ranges, mixed global/per-stream queries, skew, and high stream cardinality. They are nested conditions of one task. The evaluator fixes input lifecycle, warmup, calibration, resource limits, and invocation boundaries. Canonical children use the same two available logical CPUs, which are not dedicated. Callgrind diagnostics remain separate from native elapsed time [15].

Development comparisons contain ten complete temporal blocks. Each block includes a reference/candidate pair for every family, with five pairs in each arm order per family across the comparison. Both arms calibrate to a 0.05-second target, up to 65,536 repetitions, and use the larger count. Warmup and calibration are unscored. A completed scored sample below target remains included and flagged.

For family w and block b, define $r_{w,b} = \ln\left(\frac{t_{R,w,b}}{t_{C,w,b}}\right)$ and $S_w = \exp\big(\mathrm{median}_b\, r_{w,b}\big)$. With $W = 6$, the equal-family portfolio is

$$G = \exp\left(\frac{1}{W}\sum_{w=1}^{W} \mathrm{median}_b\, r_{w,b}\right). \tag{1}$$

Values above one favor the candidate. Ten thousand bootstrap draws jointly resample complete blocks, reusing each draw's indices across families. The portfolio lower screen L is the interpolated 10th percentile, a nominal one-sided 90% percentile-bootstrap lower bound. Promotion requires $G \geq 1.03$, $L > 1$, every $S_w \geq \frac{1}{1.10}$, and all nonperformance gates. The family floor allows at most a 10% latency increase under this estimator. Incomplete comparisons supply no estimate. Exact schedules, seeds, interpolation, technical limits, and coverage belong in the supplement.

F. Scheduling and recovery

One controller admits every arbitrary participant shell command and official native operation to a shared execution lane. Automatic development trajectories and final native assessment use the same lane. Builtin unscheduled participant shell routes are disabled; a mediated route preserves compilers, scripts, repeated experiments, and repairs within the existing sandbox. Admission is FIFO once requests reach the coordinator. A lease lasts through verified cleanup of the tracked process tree; uncertain cleanup quarantines later resource work.

Bounded structured reads remain available during timing, as do model planning and messages. They execute no shell, reject unsafe file paths and types, and expose bounds and continuation information. They avoid a native operation's metadata lock and do not themselves invalidate timing. Arbitrary shell-based edits and structured writes acquire the lane. Structured writes are confined to granted mutable scratch files, while measurements bind to immutable snapshots.

An official benchmark request contains immutable attempts tied to the same sources, binaries, fixtures, purpose, limits, procedure, and estimator. Automatic recovery requires an independently attested competing study-command overlap or specifically identified external preemption/interruption, confirmed cleanup, and unchanged identities. The reason is recorded independently of score. The whole comparison repeats; partial samples are not pooled, and retries rejoin the queue.

The first complete valid attempt ends the request regardless of score. Valid regressions, lower-bound failures, and under-

target samples never trigger recovery. Semantic or memory errors, candidate hangs, resource failures, unknown infrastructure faults, cancellation, and unresolved cleanup cannot use this route. Eligible recovery has no arbitrary attempt quota. There is no generic model-session relaunch. Waiting and invalid attempts remain in elapsed time and observed resources.

G. Factual history and stopping

A mediated journal appends factual records with identities, timestamps, role/session provenance, source hashes, and evidence links. Official outcomes and reviewer decisions come from authoritative records. Corrections append rather than overwrite earlier claims or negative results. Routine status is compact; permitted full records and ordinary evidence remain addressable through bounded retrieval.

There is no fixed campaign or reviewer wall-time cap, token cap, attempt target, or improvement quota. The lead maintains concrete investigations with motivating evidence, observable outcomes, and decision relevance. Normal completion requires a dossier closing or bounding every hypothesis, reviewer concern, failed branch, and actionable item. The reviewer checks it, reruns missing material evidence, and endorses closure or specifies further work. A runnable relevant experiment cannot be dismissed by an unsupported appeal to diminishing returns.

A final chat message does not close a campaign. Technical failure, cancellation, integrity stops, and model-session loss preserve incomplete status and the last verified incumbent, if available. No human optimization steering is planned during measured work. Any scientific or environmental intervention must be disclosed separately from the frozen controller's operations.

H. Final assessment and development trajectories

The endpoint is the last promoted source at endorsed closure, or the seed if none was promoted. A stopped campaign seals its last verified incumbent with incomplete status, or an unavailable endpoint when no incumbent is verifiable. All three endpoint seals must exist before candidate assessment or release of held-out endpoint results. Missing seals keep assessment blocked. Before timing, the evaluator checks correctness and resource compliance in both final strata and reruns Memcheck on the shared six-family public diagnostic suite. Every required check must pass; a failed endpoint is not replaced with earlier code.

Eligible changed endpoints receive fresh paired seed comparisons on a primary same-distribution/new-seed transfer portfolio and a separate secondary portfolio with changed sizes/distributions. Each contains six families and thirty complete blocks, fifteen pairs in each arm order, with the same estimator and bootstrap. Unchanged seed endpoints are labeled algebraic identity. Final assessment has no promotion threshold and cannot authorize optimization, reselection, or replacement campaigns. Its same-artifact first-valid recovery policy is unchanged. Missing and invalid estimates remain visible.

After promotion, the controller also records a direct seed-versus-checkpoint comparison on public development inputs. It reuses the selected independent comparison when the parent is the exact seed; otherwise it makes an additional ten-block comparison. Parent-relative gains are never multiplied. The primary trajectory coordinate is comparison completion time; promotion time is retained separately. The initial ratio of one denotes source identity. Usage/cost coordinates require matching authoritative coverage and are not interpolated from later receipts. Process reporting retains executed delegated work, hypothesis outcomes, reviews, repairs, rejected nominations, retries, and incomplete checkpoints.

I. Resources and reproducibility

The public Codex 0.159.3 stream supplies cumulative thread counters. Accounting counts one reconciled endpoint per unique thread and avoids charging repeated snapshots or parent/child aggregates twice. Cached input and reasoning are subsets of input and output respectively. Context bookkeeping without a cumulative increment is not new usage. Failed and interrupted work remains in scope, with unreported tails labeled unknown.

A cache-write zero may be a parser default rather than observed provider reporting [16]. Remote compaction can record usage outside the public cumulative stream [17]. Each campaign therefore reports its own coverage limits. Model-specific, dated Standard API-equivalent prices are hypothetical scenarios, with cache and price-band uncertainty preserved [18], [19]. Observed-token intervals do not bound unreported work or establish actual bills. The supplement states the formula and reconciliation rules.

Campaign elapsed time, queue delay, native CPU/wall time, setup, final assessment, and interpretation are identified separately. The artifact record retains frozen sources and instructions, source/build identities, paired measurements, attempt histories, factual journals, decisions, endpoint seals, and available usage. Technical preflight and setup controls remain separate from measured optimization outcomes.

IV. Results

A. Completion and endpoint validity

All three campaigns reached reviewer-endorsed task completion, and the automatic final assessment completed in its first invocation. The final endpoint was the last promoted source in every campaign. All 21 role threads and 68 turns reached completed status; campaign summaries reported no runtime failure. No operator optimization steering or measured-protocol changes were reported; ordinary observer activity on the shared host is disclosed in the supplement. The final evaluator rebuilt the seed and all three endpoints, obtained 89 passing correctness cases per source in each stratum, and completed all six public Memcheck probes per source. These checks establish the declared finite validation outcome. They do not prove all-input equivalence or memory safety at untested sizes.

The six final performance comparisons all used their first complete valid attempt. No final endpoint was repaired or replaced after assessment. Supplementary provenance records

bind the endpoint sources and validation coverage to the immutable report.

B. Final artifact performance

The primary transfer and separate shifted-input portfolios both favored every final endpoint (Table I; Fig. 2). Transfer speedups ranged from 2.240 to 5.445, and shifted-input speedups from 2.236 to 5.294. The lower bounds describe within-comparison timing variation and do not estimate population reliability across optimization tasks.

TABLE I
FINAL SEED-TO-ENDPOINT PORTFOLIO RESULTS

Campaign	Stratum	Speedup	L90
C1	Transfer	2.240	2.222
C1	Shift	2.236	2.214
C2	Transfer	4.547	4.507
C2	Shift	5.186	5.114
C3	Transfer	5.445	5.390
C3	Shift	5.294	5.235

Each row uses thirty paired blocks and the first complete valid attempt. L90 is the nominal one-sided 90% complete-block bootstrap lower bound. Transfer and shift are separate six-family portfolios.

The portfolio gains were not uniform. C1′s shifted sparse-family speedup was 0.911, corresponding to approximately 9.7% higher latency. Its other shifted families ranged from 1.612 to 5.271. Every transfer family improved in all three campaigns, and every shifted family improved in C2 and C3. The largest family effects occurred on mixed global/per-stream queries: transfer speedups were 4.370, 9.885, and 14.607. Complete family effects and absolute per-invocation timings appear in the supplement; these nested conditions are not additional independent tasks.

C. Investigations and development trajectories

All thirteen hypotheses had source-bound independent reviewer build, profile, and benchmark evidence. Nine closed as supported, three as contradicted, and one as inconclusive (Table II). Across the campaigns, 357 official operations included 46 benchmarks, all on first attempts. Eight nominations were promoted; none was left pending or rejected. Rejected exploratory approaches therefore must not be counted as rejected nominations.

TABLE II
CAMPAIGN PROCESS OUTCOMES

Campaign outcome	C1	C2	C3
Role threads	7	8	6
Completed turns	20	21	27
Hypotheses (S / C / I)	3/2/0	3/0/1	3/1/0
Review records	8	7	7
Source promotions	2	3	3

S / C / I: supported / contradicted / inconclusive. Review records cover claims and nominations; three closure endorsements are separate. Threads, turns, and reviews are nested process counts, not independent trials.

The direct seed-relative development checkpoints were 1.706 and 2.291 for C1; 1.284, 2.153, and 4.551 for C2; and 1.277, 2.234, and 5.596 for C3 (Fig. 3). Each point uses its own seed comparison. The curves summarize selected development checkpoints, with no performance assigned between observations and no multiplication of parent-relative effects.

The retained investigations explain why performance and hypothesis disposition differ. In C1, a contiguous requested-index proposal lost performance on sparse and dense inputs. Selective 64-event summaries reduced repeated wide-range scanning, but initially increased high-cardinality overhead; sparse allocation of summary metadata addressed that cost. An unconditional flat scatter then regressed dense inputs and remained exploratory. The final promoted source selected between vector and flat builders using a deterministic sample of current-input stream membership. Its first implementation failed eleven correctness cases with segmentation faults because a 32-bit tag truncated a size-sized sentinel. A distinct repaired source used consistent widths and passed the required checks. The failed source remains in the record.

C2 and C3 investigated query-demand pruning, shared timestamp sorting, demand-amortized block summaries, and portable radix sorting of large arrays. Their initial pruning implementations retained family losses; shared sorting removed

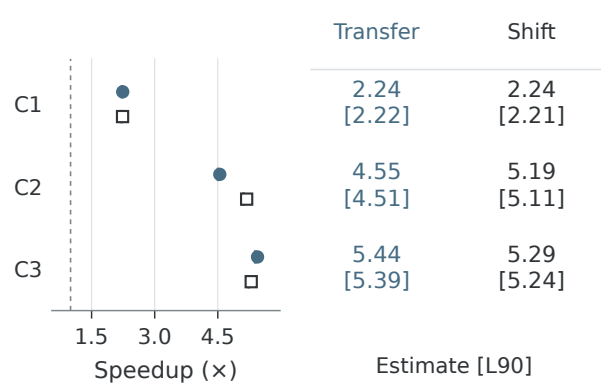


Fig. 2. Final seed-to-endpoint portfolios. Filled circles denote transfer; open squares denote shift. Points are equal-family effects and stems end at nominal one-sided 90% lower bounds (L90), from thirty paired blocks per stratum.

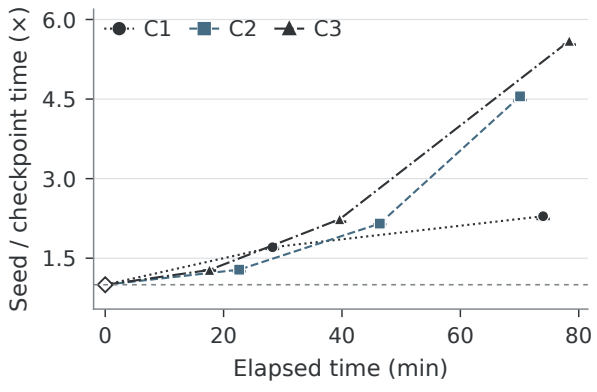


Fig. 3. Direct seed-relative development observations at comparison completion, measured from campaign-owner start. The hollow diamond at zero is unmeasured source identity. Lower stems are nominal one-sided 90% bounds for individual comparisons, not a simultaneous band.

redundant work, and summaries addressed repeated range scans. Later sources used input-derived varying timestamp bytes to skip constant radix passes. These mechanisms are supported only at their tested scope. In particular, public diagnostic inputs can fall below the large-array radix threshold, so their instruction counts alone cannot establish the cause of the final radix-related timing gains. C3′s large-path instrumentation gap at closure was later supplemented by five clean fixed-endpoint Memcheck probes. These post-hoc checks do not change original acceptance, performance, or usage records.

D. Observed resources

The runtime-host wall intervals were 84.80, 74.25, and 86.75 minutes for C1–C3. These start about eight seconds after owner initialization and include closing activity after endpoint sealing; they are not identical to source-seal intervals. Owner-start to last observed usage spanned 84.93, 74.38, and 86.89 minutes. The supplement separates bootstrap, source sealing, usage receipts, and host cleanup instead of treating a sealed source as the end of resource consumption. Final assessment added 14.47 minutes.

Observed unique-thread totals were 39.991, 44.941, and 43.954 million tokens. Input accounted for 128.066 million of the combined 128.886 million tokens; 123.561 million input tokens were reported cache reads. Repeated cached context contributes to these totals, so they are not counts of unique task content. Output already includes reasoning tokens.

A post-freeze response-band reconciliation assigned the observed cumulative increments to the short-input Standard pricing band, preserving unknown cache writes. It produced campaign subtotals of USD 5.491–5.904, 5.655–5.987, and 5.930–6.339, or USD 17.075–18.230 in total. The frozen broad scenarios remain in the supplement. C1 and C3 each showed compaction in an implementation context and a reviewer context, whose unreported usage is not recovered; C2 had no observed compaction marker. Provider request IDs, actual billing tier, and authoritative cache-write charges remain unavailable. Setup, harness authoring, interpretation, and manuscript preparation are outside this campaign subtotal. The interval neither measures an actual bill nor bounds total project cost.

V. Discussion

The observed result is that three separately initialized executions of this specified workflow produced validated faster artifacts on one synthetic task and reached reviewed closure. Transfer gains of 2.240–5.445 persisted under the separate shifted-input portfolios, but the C1 sparse regression shows why a portfolio alone is insufficient. Equal weighting permits one family's loss to coexist with a substantial aggregate gain; deployment usefulness would require the actual request distribution and absolute costs.

The process records show repeated investigation, negative dispositions, source repair, and empirical review alongside successful promotions. C1′s fault and repair illustrate a separation between a plausible mechanism and a valid implementation. C2′s inconclusive initial pruning claim illustrates that an exploratory gain need not establish the proposed general prediction. These observations characterize what happened. Without an alternative workflow arm, they cannot determine whether delegation, review, journaling, scheduling, model allocation, or unrestricted stopping caused the observed gains or was necessary to obtain them.

The three trajectories also diverged despite a common seed and contract. C2 and C3 reached radix-based endpoints, whereas C1 ended with a conditional index builder and summaries. This variation motivates testing the procedure across more tasks and baselines, including tuned implementations and matched alternative workflows. It does not rank agent designs. The observed-token accounting is useful for a reproducible resource scenario, while missing provider work and setup coverage prevent a claim of inexpensive end-to-end research.

VI. Threats to Validity

Task and baseline. One constructed engine and an untuned indexed seed provide detailed process evidence but limited coverage of software optimization. Seed-relative improvement would not establish expert competitiveness, production readiness, maintainability, security, or optimality. Equal family weights and repeated-call measurements need not represent a deployment's request distribution, cold latency, or total time saved.

Adaptive development and timing. Hypotheses, revisions, nominations, and plotted checkpoints depend on earlier development evidence. Direct seed comparisons avoid multiplicative reconstruction but remain selected observations. First-valid rules prevent score-based replacement within a bound comparison, not all development selection. Separate final inputs reduce reuse of development evidence without guaranteeing broad generalization [14]. Pairing, repetition, and a shared execution lane cannot eliminate unrelated host contention, cache and layout effects, frequency changes, or serial drift [13]. Nominal bootstrap bounds depend on block exchangeability and do not provide campaign-wide error control.

Review and semantic coverage. Reviewers and workers may share model-family errors, task interpretations, and defects in AI-assisted test or oracle construction. Independent reruns test reproducibility of material evidence but do not supply a human gold standard. Finite correctness tests, memory diagnostics, profiles, and boundary probes cannot prove all-input equivalence, race freedom, or absence of prohibited caches. Complete configured Memcheck coverage is not exhaustive branch or maximum-size coverage.

Stopping and inference. Task-based closure depends on judgments about which remaining investigations are decision-relevant. Open-ended resource use is an outcome, not a matched treatment budget. Three separately initialized campaigns share the task, seed, host, service, and design choices. They provide descriptive repeated trajectories, not precise population reliability estimates or causal component effects.

Timing blocks, workers, and the six workload families do not enlarge that campaign sample.

Information boundaries and context. Intended read scopes, disabled shell routes, structured retrieval, immutable evidence, and audit records address specific boundaries; they are not proof of hostile-code isolation or absence of every information path. Context retention and compact status are design features whose efficiency is not measured here. Any observed exposure or intervention must be described at its actual scope without inferring participant intent.

Accounting and reproducibility. Public telemetry can omit provider operations, remote-compaction usage, interrupted tails, and authoritative cache-write provenance. Price-band assumptions and missing service-tier or billing evidence limit cost claims even when observed counters reconcile. Shared-host behavior and unavailable immutable model versions limit exact replication. Frozen code and evidence support source-specific replay but cannot establish identical future agent behavior.

VII. Conclusion

Three separately initialized campaigns completed an unattended optimization workflow on one synthetic range-statistics engine. Their validated final endpoints achieved transfer portfolio speedups of 2.240, 4.547, and 5.445, while one shifted sparse condition regressed. The record retains negative hypotheses, a failed implementation and repair, direct seed-relative trajectories, and observed resource limits. The study establishes these source-specific descriptive outcomes; broader reliability, competitive performance, and the causal value of individual workflow components remain open.

VIII. Data and Code Availability

The code, frozen protocol, paired measurements, process records, and analysis scripts supporting this study will be publicly available upon publication in the accompanying Volund repository: github.com/GustavoWidman/volund. The supplement provides source hashes, detailed results, and the offline reproduction workflow.

IX. Disclosure of AI Use

OpenAI generative AI tools assisted preparation and revision of the manuscript and supporting software, analyses, and visualizations. Their use in the measured optimization campaigns is specified separately in the Method. AI assistance does not replace validation against the retained experimental evidence. The authors are responsible for the content, source attribution, and conclusions.

References

[1] A. Shypula *et al.*, "Learning Performance-Improving Code Edits," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/ce65173b994cf7c925c71b482ee14a8d-Abstract-Conference.html

[2] A. Novikov *et al.*, "AlphaEvolve: A coding agent for scientific and algorithmic discovery." [Online]. Available: https://arxiv.org/abs/2506.13131v1

[3] J. Bai, R. Xu, S. Wu, D. Yang, J. Zhao, and G. Chen, "POLO: An LLM-Powered Project-Level Code Performance Optimization Framework," in *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, International Joint Conferences on Artificial Intelligence Organization, 2025, pp. 7319–7328. doi: 10.24963/ijcai.2025/814.

[4] R. Deng *et al.*, "PerfAgent: Profiler-Guided Iterative Refinement for Repository-Level Code Optimization." [Online]. Available: https://arxiv.org/abs/2607.19653v1

[5] Q. Wu *et al.*, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversations," in *First Conference on Language Modeling*, 2024. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2023/08/LLM_agent.pdf

[6] S. Hong *et al.*, "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://proceedings.iclr.cc/paper_files/paper/2024/hash/6507b115562bb0a305f1958ccc87355a-Abstract-Conference.html

[7] C. Qian *et al.*, "ChatDev: Communicative Agents for Software Development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2024, pp. 15174–15186. doi: 10.18653/v1/2024.acl-long.810.

[8] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-Agent-based Code Generation with Iterative Testing and Optimisation." [Online]. Available: https://arxiv.org/abs/2312.13010

[9] X. He *et al.*, "SWE-Perf: Can Language Models Optimize Code Performance on Real-World Repositories?," in *Proceedings of the 43rd International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 306. PMLR, 2026, pp. 41651–41666. [Online]. Available: https://proceedings.mlr.press/v306/he26y.html

[10] M. Shetty, N. Jain, J. Liu, V. Kethanaboyina, K. Sen, and I. Stoica, "GSO: Challenging Software Optimization Tasks for Evaluating SWE-Agents," in *Advances in Neural Information Processing Systems*, 2025. doi: 10.52202/085713-5054.

[11] D. Grubisic, C. Cummins, V. Seeker, and H. Leather, "Compiler generated feedback for Large Language Models." [Online]. Available: https://arxiv.org/abs/2403.14714v1

[12] T. Kalibera and R. Jones, "Rigorous Benchmarking in Reasonable Time," in *Proceedings of the 2013 International Symposium on Memory Management*, in ISMM '13. ACM, 2013, pp. 63–74. doi: 10.1145/2464157.2464160.

[13] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing Wrong Data Without Doing Anything Obviously Wrong!," in *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, in ASPLOS '09. ACM, 2009, pp. 265–276. doi: 10.1145/1508244.1508275.

[14] G. C. Cawley and N. L. C. Talbot, "On Over-fitting in Model Selection and Subsequent Selection Bias in Performance Evaluation," *Journal of Machine Learning Research*, vol. 11, no. 70, pp. 2079–2107, 2010, [Online]. Available: https://www.jmlr.org/papers/v11/cawley10a.html

[15] Valgrind Developers, "Callgrind: A Call-Graph Generating Cache and Branch Prediction Profiler." [Online]. Available: https://valgrind.org/docs/manual/cl-manual.html

[16] OpenAI, "Codex Response Usage Parser." [Online]. Available: https://github.com/openai/codex/blob/rust-v0.159.3/codex-rs/codex-api/src/sse/responses.rs#L138

[17] OpenAI, "Codex Remote Compaction and Usage Accounting." [Online]. Available: https://github.com/openai/codex/blob/rust-v0.159.3/codex-rs/core/src/compact_remote_v2.rs#L223

[18] OpenAI, "OpenAI API Pricing." 2026. [Online]. Available: https://developers.openai.com/api/docs/pricing

[19] OpenAI, "Prompt Caching." 2026. [Online]. Available: https://developers.openai.com/api/docs/guides/prompt-caching